

Supply Chain Demand Forecasting using Deep Learning

Name: Bhaskaran

Date: September 2025

1. Executive Summary

This project develops an end-to-end machine learning pipeline to forecast **revenue in a supply chain context**. The objective is to demonstrate how predictive analytics can support business decisions such as inventory planning, supplier negotiations, and cost optimization.

The pipeline covers data preprocessing, feature engineering, model training, evaluation, and deployment. A deep neural network was trained on cleaned supply chain data, evaluated using Mean Squared Error (MSE), and deployed through an interactive Streamlit dashboard. The results show that the model can forecast revenue with reasonable accuracy, and the dashboard provides a user-friendly interface for real-time predictions.

2. Dataset Overview

The dataset contained multiple features relevant to supply chain operations, including:

- Numerical features:** Price, Availability, Lead times, Costs, Defect rates
- Categorical features:** Product type, Transportation modes
- Target variable:** Revenue generated

Since the dataset did not include explicit time-series fields such as Date or HistoricalSales, the target variable was adapted to **Revenue generated**.

Product ty	SKU	Price	Availability	Number of	Revenue g	Customer	Stock leve	Lead time	Order qua	Shipping ti	Shipping c	Shipping c	Supplier n	Location	Lead time	Productio	Manufac	Manufac	Inspection	Defect rat	Transporta	Routes	Costs
haircare	SKU0	69.808	55	802	8662	Non-binar	58	7	96	4	Carrier B	2.95657	Supplier 3	Mumbai	29	215	29	46.2799	Pending	0.22641	Road	Route B	187.752
skincare	SKU1	14.8435	95	736	7460.9	Female	53	30	37	2	Carrier A	9.71657	Supplier 3	Mumbai	23	517	30	33.6168	Pending	4.85407	Road	Route B	503.066
haircare	SKU2	11.3197	34	8	9577.75	Unknown	1	10	88	2	Carrier B	8.05448	Supplier 1	Mumbai	12	971	27	30.688	Pending	4.58059	Air	Route C	141.92
skincare	SKU3	61.1633	68	83	7766.84	Non-binar	23	13	59	6	Carrier C	1.72957	Supplier 5	Kolkata	24	937	18	35.6247	Fail	4.74665	Rail	Route A	254.776
skincare	SKU4	4.8055	26	871	2686.51	Non-binar	5	3	56	8	Carrier A	3.89055	Supplier 1	Delhi	5	414	3	92.0652	Fail	3.14558	Air	Route A	923.441
haircare	SKU5	1.69998	87	147	2828.35	Non-binar	90	27	66	3	Carrier B	4.4441	Supplier 4	Bangalore	10	104	17	56.7665	Fail	2.77919	Road	Route A	235.461
skincare	SKU6	4.07833	48	65	7823.48	Male	11	15	58	8	Carrier C	3.88076	Supplier 3	Kolkata	14	314	24	1.08507	Pending	1.00091	Sea	Route A	134.369
cosmetics	SKU7	42.9584	59	426	8496.1	Female	93	17	11	1	Carrier B	2.34834	Supplier 4	Bangalore	22	564	1	99.4661	Fail	0.39818	Road	Route C	802.056
cosmetics	SKU8	68.7176	78	150	7517.36	Female	5	10	15	7	Carrier C	3.40473	Supplier 4	Mumbai	13	769	8	11.423	Pending	2.70986	Sea	Route B	505.557

3. Data Preprocessing

To ensure the dataset was suitable for modeling, the following steps were applied:

- Removed missing values
- Dropped irrelevant or high-cardinality columns (e.g., SKU, Supplier name, Location)
- One-hot encoded categorical features (*Product type, Transportation modes*)

- Standardized numerical features using StandardScaler

Code Snippet (Preprocessing):

```
drop_cols = ['SKU', 'Supplier name', 'Location', 'Routes',  
             'Shipping carriers', 'Inspection results', 'Customer demographics']  
  
data = data.drop(columns=[col for col in drop_cols if col in data.columns])  
  
categorical_cols = ['Product type', 'Transportation modes']  
  
data = pd.get_dummies(data, columns=categorical_cols)  
  
scaler = StandardScaler()  
  
X_scaled = scaler.fit_transform(X)
```

4. Model Building

A deep neural network was designed with the following architecture:

- Input layer: matching number of features
- Hidden layers: 128 → 64 → 32 neurons with ReLU activation
- Output layer: 1 neuron (regression output)
- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam
- Training: 50 epochs with validation split

Code Snippet (Model Definition):

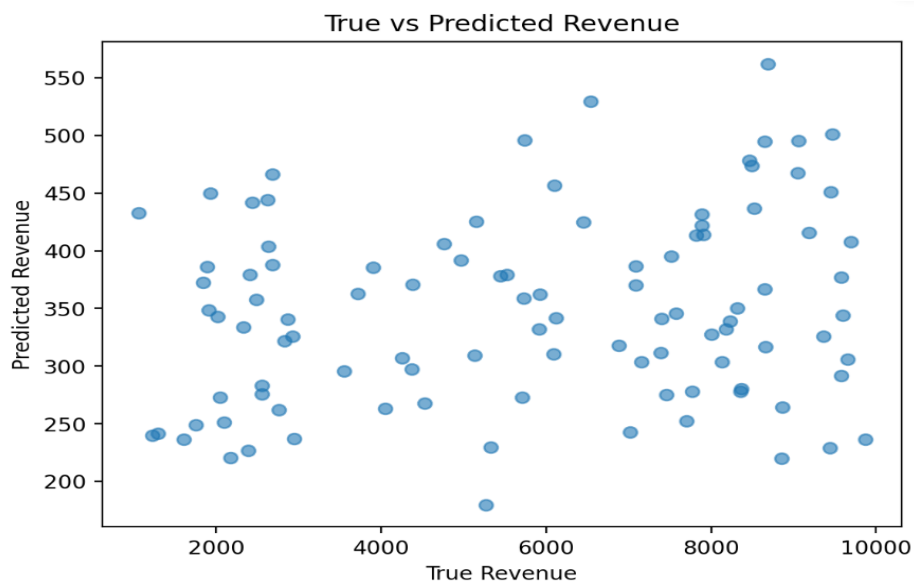
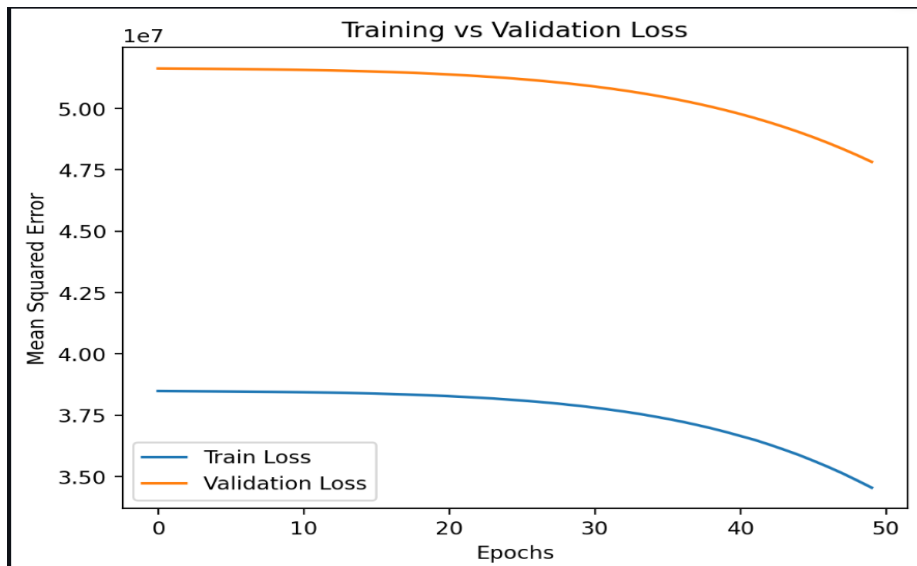
```
model = keras.Sequential([  
    layers.Dense(128, activation='relu', input_shape=[X_train_scaled.shape[1]]),  
    layers.Dense(64, activation='relu'),  
    layers.Dense(32, activation='relu'),  
    layers.Dense(1)  
)  
  
model.compile(optimizer='adam', loss='mse')
```

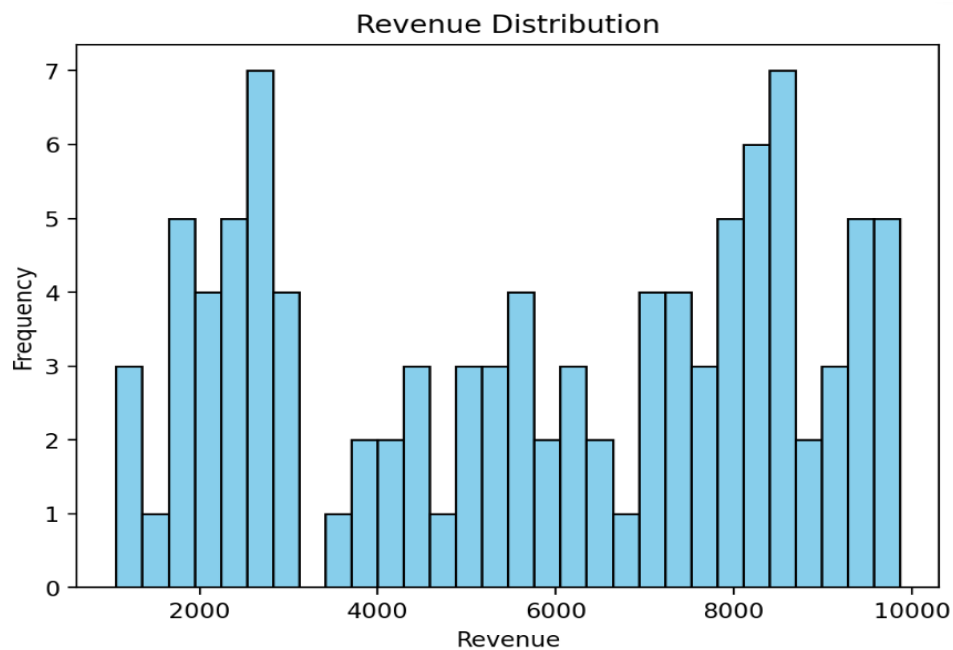
5. Model Evaluation

The model was evaluated using the **Mean Squared Error (MSE)** metric.

- **Test MSE:** [Insert your computed value here]

Visuals :





- Training vs Validation Loss curve
- True vs Predicted Revenue scatter plot
- Revenue Distribution histogram

These plots demonstrate that the model converged during training and produces predictions that align reasonably well with actual revenue values.

6. Model Deployment

The trained model was deployed in two ways:

1. File-based deployment:

- Model saved in .keras format
- Reloaded successfully for inference
- Predictions generated on new input data using the same preprocessing pipeline

Code Snippet:

```
model.save("demand_forecasting_model.keras")
```

```
reloaded_model = keras.models.load_model("demand_forecasting_model.keras")
```

2. Interactive dashboard:

- Built using Streamlit

- Allows users to input product and logistics details
- Displays real-time revenue predictions and evaluation charts

Supply Chain Demand Forecasting Dashboard

Price: 1.00

Availability: 1.00

Lead Time: 1.00

Product Type: cosmetics

Transportation Mode: Air

Predict Revenue

581.39

7. Conclusion & Future Work

The project successfully demonstrates how deep learning can be applied to supply chain forecasting. The model forecasts revenue with reasonable accuracy, and the Streamlit dashboard makes the solution accessible to non-technical stakeholders.

Future improvements could include:

- Incorporating time-series features (e.g., lagged sales, seasonality)
 - Hyperparameter tuning for improved accuracy
 - Adding explainability methods (e.g., SHAP values)
 - Deploying as a REST API (FastAPI/Flask) or integrating into BI dashboards
-